Universal Asynchronous RX/TX

Department of Electrical Electronic Engineering,
UTP

3/17/2014

Adapted from

Richard Kuo
Assistant Professor
National Taiwan Ocean University

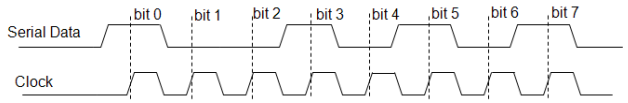
Introducing Serial Data Communication

In serial data transfer, one bit of the data is transmitted at a time, along a single interconnection.

While slower than parallel transfer, the small number of wires needed is a huge advantage, especially in the embedded world, i.e. less pcb tracks, interconnecting wires, and i.c. pins.

Once we start applying the serial concept, a number of challenges arise. How does the receiver know when each bit begins and ends, and how does it know when each word begins and ends?

One way is to send a clock signal alongside the data, with one clock pulse per data bit. The data is *synchronised* to the clock. This is called synchronous serial communication.

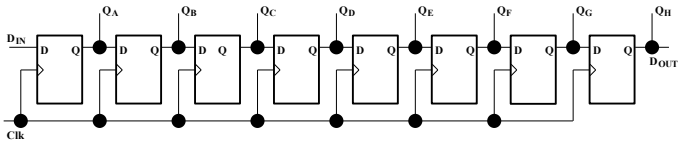


Synchronous serial data

The Basics of a Serial Port: the Shift Register

An essential feature of most serial links is a *shift register.* This is made up of a string of flip-flops, connected so that the output of one is connected to the input of the next. Each flip-flop holds one bit of information.

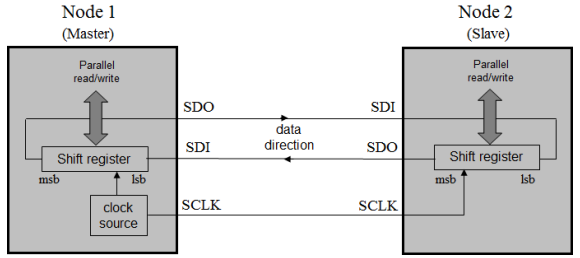
Every time the shift register is pulsed by the clock signal, each flip-flop passes its bit on to its neighbour on one side, and receives a new bit from its other neighbour. The one at the input end clocks in data received from the outside world, and the one of the output end outputs its bit.



An 8-Bit Shift Register – a Possible Receiver and/or Transmitter of Serial Data

A Simple Serial Link

A simple (synchronous) serial data link is shown. Node 1 is designated *Master*; it controls what's going on, as it controls the clock. The *Slave* is similar to the Master, but receives the clock signal from the Master.



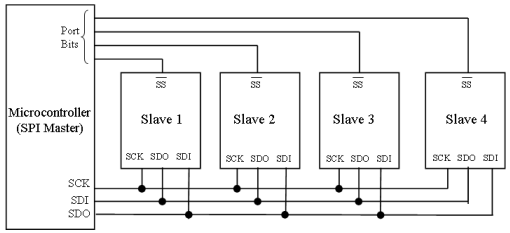
A simple Serial Link

Serial Peripheral Interface (SPI)

In the early days of microcontrollers, both National Semiconductors and Motorola started introducing simple serial communication, based on the previous Figure.

Each formulated a set of rules which governed how these links worked, and allowed others to develop devices which could interface correctly.

These became *de facto* standards. Motorola called its standard *Serial Peripheral Interface (SPI)*, and National Semiconductors called theirs *Microwire.* They're very similar to each other.



SPI interconnections for multiple Slaves

Evaluating SPI

The SPI standard is extremely effective. The electronic hardware is simple and therefore cheap, and data can be transferred rapidly.

There are disadvantages.

- There is no acknowledgement from the receiver, so in a simple system the Master cannot be sure that data has been received.
- There is no addressing. In a system where there are multiple slaves, a separate |SS line must be run to each Slave, as seen earlier. Therefore we begin to lose the advantage that serial communications should give us, i.e. a limited number of interconnect lines.
- There is no error-checking. Suppose some electromagnetic interference was experienced in a long data link, data or clock would be corrupted, but the system would have no way of detecting this, or correcting for it.

Overall SPI could be evaluated as simple, convenient and low-cost, but not appropriate for complex or high reliability systems.

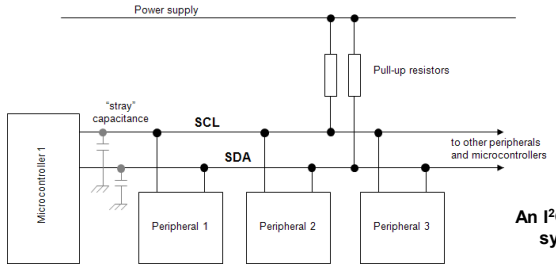
Introducing I²C

The name I²C stands for Inter-Integrated Circuit bus. It aims to resolve some of the perceived weaknesses of SPI.

I²C is a serial data protocol which operates with a master/slave relationship.

I²C only uses two physical wires, called serial data (SDA) and serial clock (SCL). This means that data only travels in one direction at a time.

Any node can only pull down the SCL or SDA line to Logic 0; it cannot force the line up to Logic 1. This role is played by a single pull-up resistor connected to each line.



An I²C-based system

18

8

Simple I²C Communications

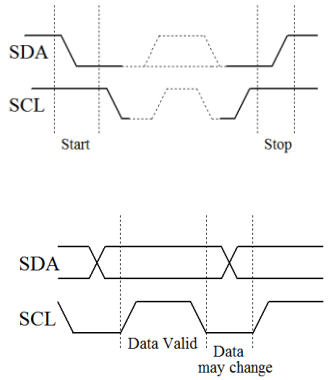
The device that initiates communication is termed the 'master'. A device being addressed by the master is called a 'slave'.

A data transfer is started by the master signalling a Start condition, followed by one or two bytes containing address and control information.

The Start condition is defined by a high to low transition of SDA when SCL is high.

A low to high transition of SDA while SCL is high defines a Stop condition

One SCL clock pulse is generated for each SDA data bit, and data may only change when the clock is low.



Start and Stop conditions

9

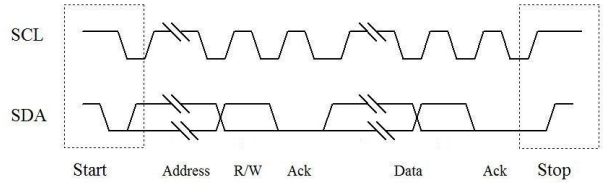
Simple I²C Communications

I²C has a built-in addressing scheme, which simplifies the task of linking multiple devices together. Each slave has a predefined address. Slaves monitor the bus and respond only to data and commands associate with their own address.

The byte following the Start condition is made up of seven address bits, and one data direction bit (Read/Write).

All data transferred is in units of one byte, with no limit on the number of bytes transferred in one message.

Each byte must be followed by a 1-bit acknowledge from the receiver, during which time the transmitter relinquishes SDA control.



A Complete transfer of one byte

Evaluating synchronous serial data communication

Synchronous serial communication protocols (like SPI, I2C) are extremely useful ways of moving data around. But taking a clock signal to every node has these disadvantages:

• An extra (clock) line needs to go to every data node.

• The bandwidth needed for the clock is always twice the bandwidth needed for the data; therefore, it is the demands of the clock which limit the overall data rate.

• Over long distances, clock and data themselves could lose synchronisation.

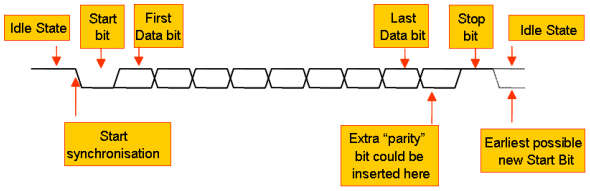
Asynchronous serial data communication

Asynchronous communication doesn't require the clock to be connected between nodes. A common approach is:

• Data rate is predetermined – both transmitter and receiver are pre-set to recognise the same data rate. Hence each node needs an accurate and stable clock source, from which the data rate can be generated. Small variations from the theoretical value can however be accommodated.

• Each byte or word is *framed* with a Start and Stop bit. These allow synchronisation to be initiated before the data starts to flow.

• An asynchronous serial port is generally called a UART, *Universal Asynchronous Receiver/ Transmitter.* A UART has one connection for transmitted data, called TX, and another for received data, called RX. The data rate that receiver and transmitter will operate at must be pre-determined; this is specified by its *baud rate*.



UART pins

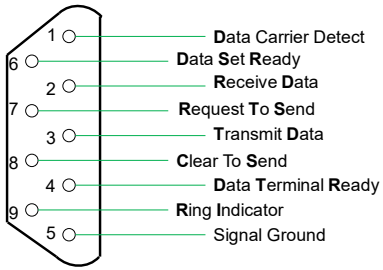
Based on NUC140 LQFP 100pin

- UART0
  - RX0/GPB0 : pin32
  - TX0/GPB1 : pin33
- UART1
  - RX1/GPB4 : pin19
  - TX1/GPB5 : pin20
- UART2
  - RX2/GPD14 : pin38
  - TX2/GPD15 : pin39

UART Baud Rate : Cycle Time & Interrupt

Baud Rate	Cycles per Timer Tick (at 3.57MHz)	Worst-Case Interrupt Cycle Count	Bandwidth Used by UART (%)
9600	372	19	5.1%
19200	186	19	10.2%
28800	124	19	15.3%
57600	62	19	31.6%
115200	31	19	61.3%

RS-232 DB9 Connector



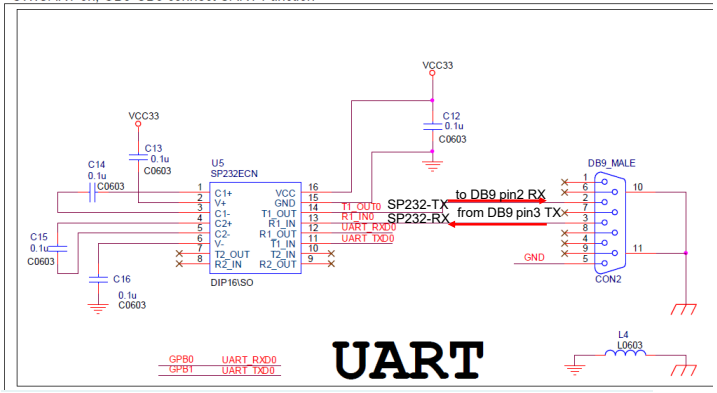
RS-232/422/485 Comparison

	RS-232	RS-422	RS-485
No. of Driver	1	1	32
No. of Receiver	1	10	32
Mode of Operation	Half duplex Full duplex	Half duplex	Half duplex
Max. Distance	15m	1200m	1200m
Max. Speed at 12m	20Kbps	10Mbps	35Mbps
Max. Speed at 1200m	1Kbps	100Kbps	100Kbps
Min. Output Voltage	+-5V	+-2.0V	+-1.5V
Max. Output Voltage	+-25V	+-6V	-7~12V
Receiver Input Range	+-15V	+-10V	-7~12V

UART schematic

UART

SW:UART on, GB0-GB3 connect UART Function



NuMicro MCU UART function

- Provide up to 3 UART ports
  - programmable baud-rate
  - programmable serial-interface
  - Built-in TX/RX buffers (FIFO)
    - UART0 FIFO = 64 bytes
    - UART1 FIFO = 16 bytes
    - UART2 FIFO = 16 bytes
- Support hardware flow-control (/CTS及/RTS) on UART0 & UART1
- Support IrDA SIR function (Infrared transmission use)

UART0 initialization example


```
int32_t main (void)
{
    STR_UART_T param;
    /* GPIO initial */
    DrvGPIO_InitFunction(FUNC_UART0);

     /* Enable and Select UART clock source*/
    UNLOCKREG();
    /* Enable the 12MHz oscillator oscillation */
    DrvSYS_SetOscCtrl(E_SYS_XTL12M, 1);
    /* Waiting for 12M Xtal stalble */
    SysTimerDelay(5000);
    /* HCLK clock source. 0: external 12MHz; 4:internal 22MHz RC oscillator */
    DrvSYS_SetHCLKSource(0);
    LOCKREG();

    /*Open UART0*/
    param.u32BaudRate        = 115200;
    param.u8cDataBits        = DRVUART_DATABITS_8;
    param.u8cStopBits        = DRVUART_STOPBITS_1;
    param.u8cParity          = DRVUART_PARITY_NONE;
    param.u8cRxTriggerLevel  = DRVUART_FIFO_1BYTES;
    param.u8TimeOut          = 0;
    DrvUART_Open(UART_PORT0, &param);
```

Smpl_UART0

- TX at main loop
- RX at UART interrupt handler
- UART connection (digital I/O without transceiver)
  - UART-RX0 connected to another board's TX0
  - UART-TX0 connected to another board's RX0

Smpl_UART0


```
//
// Smpl_UART0 : while loop for UART0-TX keep transmitting 8 bytes string
//            : IRQ routine for UART0-RX keep receiving 8 bytes string & print to LCD
//            : need two learning board to perform UART communication (TX & RX at the same time)
//
// Nu-LB-NUC140
// pin32 GPB0/RX0 to another board's UART TX
// pin33 GPB1/TX0 to another board's UART RX

#include <stdio.h>
#include "NUC1xx.h"
#include "GPIO.h"
#include "SYS.h"
#include "UART.h"
#include "LCD.h"
volatile uint8_t comRbuf[16];
volatile uint16_t comRbytes = 0;
volatile uint16_t comRhead  = 0;
volatile uint16_t comRtail  = 0;

char TEXT1[16] = "TX: sending...  ";
char TEXT2[16] = "RX:             ";
```

Smpl_UART0


```
void UART_INT_HANDLE(void)
{
        uint8_t i;
        uint8_t bInChar[1] = {0xFF};

        while(UART0->ISR.RDA_IF==1)
        {
                DrvUART_Read(UART_PORT0,bInChar,1);
                if(comRbytes < 8) // check if Buffer is full
                {
                        comRbuf[comRbytes] = bInChar[0];
                        comRbytes++;
                }
                else if (comRbytes==8)
                {
                        comRbytes=0;
                        sprintf(TEXT2+4,"%s",comRbuf);
                        print_Line(2,TEXT2);
                }
        }
}
```


Smpl_UART0


```
int32_t main()
{
  uint8_t  i =0;
  uint8_t  dataout[9] = "NuMicro0";
  STR_UART_T sParam;

  UNLOCKREG();
  DrvSYS_Open(48000000);
  LOCKREG();
  init_LCD();
  clear_LCD();
  print_Line(0,"Smpl_UART0    ");

  /* Set UART Pin */
  DrvGPIO_InitFunction(E_FUNC_UART0);

  /* UART Setting */
  sParam.u32BaudRate      = 9600;
  sParam.u8cDataBits      = DRVUART_DATABITS_8;
  sParam.u8cStopBits      = DRVUART_STOPBITS_1;
  sParam.u8cParity        = DRVUART_PARITY_NONE;
  sParam.u8cRxTriggerLevel= DRVUART_FIFO_1BYTES;

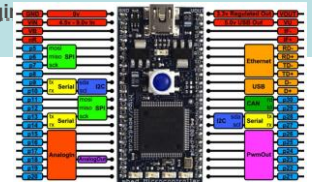
  /* Set UART Configuration */
  if(DrvUART_Open(UART_PORT0,&sParam) !=
E_SUCCESS);
  DrvUART_EnableInt(UART_PORT0, DRVUART_RDAINT,
UART_INT_HANDLE);
  while(1)
  {
    dataout[7] = 0x30 +i;
    DrvUART_Write(UART_PORT0, dataout,8);
    i++;
    if (i>=10) i = 0;
    sprintf(TEXT1+14,"%d",i);
    print_Line(1,TEXT1);
    DrvSYS_Delay(5000000);
  }
  //DrvUART_Close(UART_PORT0);
}
```

Important Notice !

- **This educational material are neither intended nor warranted for usage in systems or equipment, any malfunction or failure of which may cause loss of human life, bodily injury or severe property damage. Such applications are deemed, "Insecure Usage".**
- **Insecure usage includes, but is not limited to: equipment for surgical implementation, atomic energy control instruments, airplane or spaceship instruments, the control or operation of dynamic, brake or safety systems designed for vehicular use, traffic signal instruments, all types of safety devices, and other applications intended to support or sustain life.**
- **All Insecure Usage shall be made at user's own risk, and in the event that third parties lay claims to the author as a result of customer's Insecure Usage, the user shall indemnify the damages and liabilities thus incurred by using this material.**
- **Please note that all lecture and sample codes are subject to change without notice. All the trademarks of products and companies mentioned in this material belong to their respective owners.**

Mbed LPC1768 UART

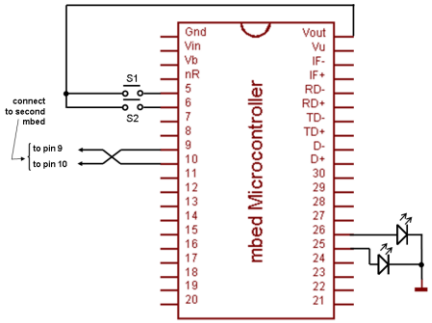
The LPC1768 has *four* UARTs. *Three* of these link to the mbed pins, simply labelled "Serial". The API summary is shown.



Functions	Usage
Serial	Create a Serial port, connected to the specified transmit and receive pins
baud	Set the baud rate of the serial port
format	Set the transmission format used by the Serial port
putc	Write a character
getc	Read a character
printf	Write a formatted string
scanf	Read a formatted string
readable	Determine if there is a character available to read
writeable	Determine if there is space available to write a character
attach	Attach a function to call whenever a serial interrupt is generated

Bidirectional data transfer between two mbed UARTs: Circuit

The TX from one mbed connects to the RX on the other mbed board



- While the mbed has three UARTs connectected to the external pins, the LPC1768 has a fourth. This is reserved for communication back to the USB link.
- The UART acts like any of the others in terms of its use of the API. The compiler recognises pc , USBTX, USBRX as identifiers to set up this connection.

```
Serial pc(USBTX, USBRX);
```

- With PC created and the API functions can be used.

